# TP N°3
# Vers un réseau plus réel

## 1. Construction de la topologie

Nous voulons obtenir la topologie suivante :

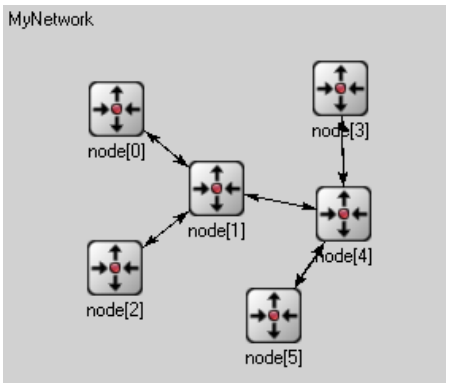



**Figure 1 Topologie demandée**

### Déclaration des nœuds

Dans un réseau réel, le nombre de nœuds peut être important et la déclaration de ces nœuds un à un dans le fichier NED devient peu pratique. Une méthode intéressante est de déclarer un tableau de nœuds avec le nombre de nœud comme taille de ce tableau comme suit :

```
network Reseau
{
    @display("bgb=362,240");
    submodules:
        noeud[6]: Noeud {
        }
```

La déclaration d'un tableau nœud[6] veut dire qu'il y'a 6 nœuds dans le réseau (cmme sur la figure 1) (nœud[0],nœud[1], nœud [2], nœud[3], nœud[4], nœud[5]). Les nœuds peuvent être identifiés grâce à leurs indexes en utilisant la fonction **getIndex()** dans le fichier C.

### Connexion d'un nœud avec plusieurs autres nœuds

Pour qu'un nœud puisse se connecter avec plusieurs nœuds, il faut déclarer un tableau de passerelles dans le fichier NED. On peut utiliser les passerelles de type input et output vues précédemment ou bien utiliser une passerelle de type inout qui joue le rôle des deux comme suit :

```
simple Noeud
{
    gates:
        inout p[];

}
```

# TP N°3
# Vers un réseau plus réel

Pour connecter les nœuds entre eux selon la topologie de la figure 1, il faut rajouter ceci :

```
connections:
    noeud[0].p++ <--> noeud[1].p++;
    noeud[1].p++ <--> noeud[2].p++;
    noeud[1].p++ <--> noeud[4].p++;
    noeud[4].p++ <--> noeud[5].p++;
    noeud[4].p++ <--> noeud[3].p++;
```

Quand un nœud veut envoyer un message via une passerelle vers un nœud spécifique, il faut qu'il spécifie quelle passerelle et quel noeud sont concernés. Pour faire ceci, il faut utiliser la fonction Send avec les trois paramètres (le nom du message, le nom de la passerelle et le numéro de passerelle) comme suit :

```
send(msg,"p$o",k);
```

Quand un nœud veut savoir de quel numéro de passerelle un message est reçu, il faut utiliser les deux fonctions suivantes :

```
cGate *g=msg->getArrivalGate();
EV<<"receiving msg from "<<g->getIndex()<<endl;
```

Pour connaitre le nombre de passerelles, il faut utiliser la fonction **gateSize(type_passerelle).**
int n = gateSize("out"); => n contient le nombre de passerelles de sortie
int n = gateSize("in"); => n contient le nombre de passerelles d'entrée
Dans le cas d'une passerelle inout :
int n = gateSize("p$o"); => n contient le nombre de passerelles de sortie
int n = gateSize("p$i"); => n contient le nombre de passerelles d'entrée

## 2. Utilisation des messages personnalisés

Pour qu'un nœud puissent puisse échanger des informations personnalisées avec les autres nœuds, il faut définir la structure du message personnalisé en créant un fichier MSG (Message Definition).




**Figure 2 Structure du message personnalisé**

Si on veut par exemple, échanger des messages contenant le jour, le mois et l'année comme décrit sur la figure 2 , il faut créer un fichier appelé par exemple Hello.msg et taper le code suivant :

```
message Hello
{int jour;
 string mois;
 int annee;
}
```

# TP N°3
# Vers un réseau plus réel

Le code ci-dessus définit la structure du message personnalisé appelé Hello.

Suite à la compilation du projet, omnetpp va générer automatiquement deux fichiers : Hello_m.h et Hello_m.c. Ces deux fichiers définissent une classe appelé Hello (liée au message Hello) avec les attributs définis dans le fichier MSG et les méthodes nécessaires à la manipulation de cette classe (constructeurs, destructeurs, getteurs, setteurs..) . Cette classe peut ainsi être utilisée dans le fichier C de la même façon que la classe CMessage.

### Exploiter le contenu du message reçu

Quand un message est reçu au niveau de la méthode HandleMessage(), il est de type cMessage même si le nœud émetteur a envoyé un message de type Hello par exemple.

Donc, le nœud récepteur ne peut pas exploiter directement le message de type Hello pour extraire le jour, le mois et l'année. Il doit passer par le Cast comme suit :

```
Hello *msg_received = check_and_cast<Hello *>(msg);
```

### 3. Utilisation la méthode finish()

La méthode finish() est invoquée à la fermeture de la fenêtre de simulation et sert à afficher les résultats de la simulation après sa terminaison. On peut par exemple afficher le nombre de messages reçus pour chaque nœud.

### Scénario à réaliser

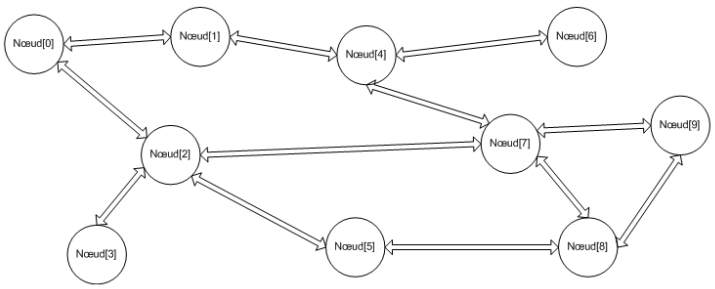



**Figure 3 Topologie avec 10 nœuds**

Soit la topologie représentée dans la figure 3. On souhaite implémenter le routage d'un message à partir du nœud 0 appelé nœud source jusqu'au nœud 9 appelé nœud destination. Le nœud 0 initie le message et l'envoi aléatoirement à l'un de ses voisins. Chaque voisin qui reçoit le message vérifie si c'est lui le destinataire du message. Si oui, il affiche à l'écran combien de sauts le message a effectué avant d'arriver chez lui sinon il transfère le message à l'un de ses voisins aléatoirement.

1. Implémenter et simuler le scénario décrit ci-dessus.
2. Quel est l'inconvénient de ce routage ?
3. Proposer une solution qui améliore ce routage.